

Un sistem de operare folosește paginarea memoriei, lucrând cu pagini de 4KB. Spațiul virtual de adrese al fiecărui proces este de 128KB. Sistemul de operare rulează pe un calculator cu memoria RAM de 512KB.

Se dă tabela de paginare pentru un proces P, cu corespondența dintre numărul unei pagini din memoria virtuală și numărul cadrului din memoria fizică. Se consideră că paginile care nu apar în tabela de mai jos nu sunt mapate.

Număr pagină	Număr cadru
0	2
5	8
6	7
8	5

Numărul de pagini din memoria virtuală a fiecărui proces este: ✓ .

Numărul de cadre din memoria fizică a sistemului este: ✓ .

Adresele virtuale în acest sistem de operare se pot reprezenta pe ✗ [17] biți.

Completați mai jos adresa fizică corespunzătoare fiecărei adrese virtuale date. În cazul în care o adresă virtuală nu este mapată completați valoarea -1.

Adresa virtuală	Adresa fizică
35050	22762 ✓
25189	29285 ✓
22339	34627 ✓
13548	-1 ✓

Se execută programul de mai jos, fiind inițializat cu

```
int n=6; int v[]={2, 8, 6, 3, 5, 9};
```

```
int sum = 0;
for(int i=0; i<n; i++) {
    if(fork() == 0) {
        sum += v[i];
        if(v[i] % 3 == 0) {
            exit(0);
        }
    } else {
        sum += 12;
        if(v[i] % 4 == 0) {
            exit(0);
        }
    }
}
printf("sum = %d\n", sum);
```

Variabila `sum` este afișată (la ultima linie) de  #4# ori.

Valoarea minimă a sumei afișate este  #51#.

Valoarea maximă a sumei afișate este  #68#.

Se rulează codul de mai jos:

```
int i = 0, pos = 0;
unsigned short magic = 0;
unsigned int x = 0;
unsigned char start = 0, c = 0;
int fd = open("test.dat", O_RDONLY);
while(read(fd, &c, 1) == 1) {
    printf("%02x ", c);
    if((++i) % 16 == 0) {
        printf("\n");
    }
}
printf("\n");
lseek(fd, -3, SEEK_END);
read(fd, &magic, sizeof(magic));
printf("magic nibble = [%d]\n", magic % 16);
read(fd, &start, sizeof(start));
pos = lseek(fd, start, SEEK_SET);
printf("pos = [%d]\n", pos);
do {
    if(read(fd, &c, 1) != 1) break;
} while(c > 0);
lseek(fd, -2, SEEK_CUR);
read(fd, &x, 4);
printf("x = [%d]\n", x);
read(fd, &c, 1);
printf("c = [%02x]\n", c);
read(fd, ((unsigned char*)&x + 1), 1);
printf("x = [%d]\n", x);
```

Completați rezultatul afișat pe ecran.

```
4e 4e 4b 58 46 56 4b 47 4e 59 52 4b 46 5a 4b 48
56 47 57 54 48 49 51 52 48 41 35 00 00 00 46 01
4c 59 48 4c 4b 4c 57 42 59 54 59 4c 49 44 4f 58
41 13
```

magic nibble = ✓

pos = ✗ [19]

x = ✓

c = ✗ [46]

x = ✗ [309]

În pseudocodul de mai jos, se consideră inițializările

$n = 7$

$v = \{2, 0, 1, 4, 3, 6, 5\}$

Se consideră că indicii din vector încep de la 1.

Funcțiile `MUTEX-LOCK` și `MUTEX-UNLOCK` realizează operațiile respective pe lacăte (mutex-uri).

Funcțiile `CV-WAIT` și `CV-BROADCAST` realizează operațiile respective pe variabile condiționale.

Funcția `CREATE-THREAD` crează un thread nou, care primește ca prim argument funcția thread-ului iar restul argumentelor se transmit funcției thread-ului.

<code>THREAD-COUNTER()</code>	<code>THREAD-ALL(a_1, a_2, a_3)</code>
1 while TRUE	1 <code>MUTEX-LOCK(mutex)</code>
2 <code>MUTEX-LOCK(mutex)</code>	2 while $val \neq a_1$
3 $val = (val + 1) \bmod n$	3 <code>CV-WAIT(cond, mutex)</code>
4 <code>CV-BROADCAST(cond)</code>	4 while $val \neq a_2$
5 <code>MUTEX-UNLOCK(mutex)</code>	5 <code>CV-WAIT(cond, mutex)</code>
6 sleep 1 second	6 while $val \neq a_3$
7 return TRUE	7 <code>CV-WAIT(cond, mutex)</code>
	8 <code>MUTEX-UNLOCK(mutex)</code>
	9 return TRUE
1 $val = -1$	
2 $mutex = \text{MUTEX}()$	
3 $cond = \text{CONDITION-VARIABLE}()$	
4 for $i = 1$ to $n - 2$	
5 // thread number i	
6 <code>CREATE-THREAD(THREAD-ALL, $v[i], v[i + 1], v[i + 2]$)</code>	
7 <code>CREATE-THREAD(THREAD-COUNTER)</code>	

Pentru fiecare thread ce rulează funcția `THREAD-ALL` (de la 1 la $n-2$), calculați timpul total de rulare (în secunde).

- 1: ✖ [8]
2: ✖ [4]
3: ✖ [10]
4: ✖ [13]
5: ✖ [12]

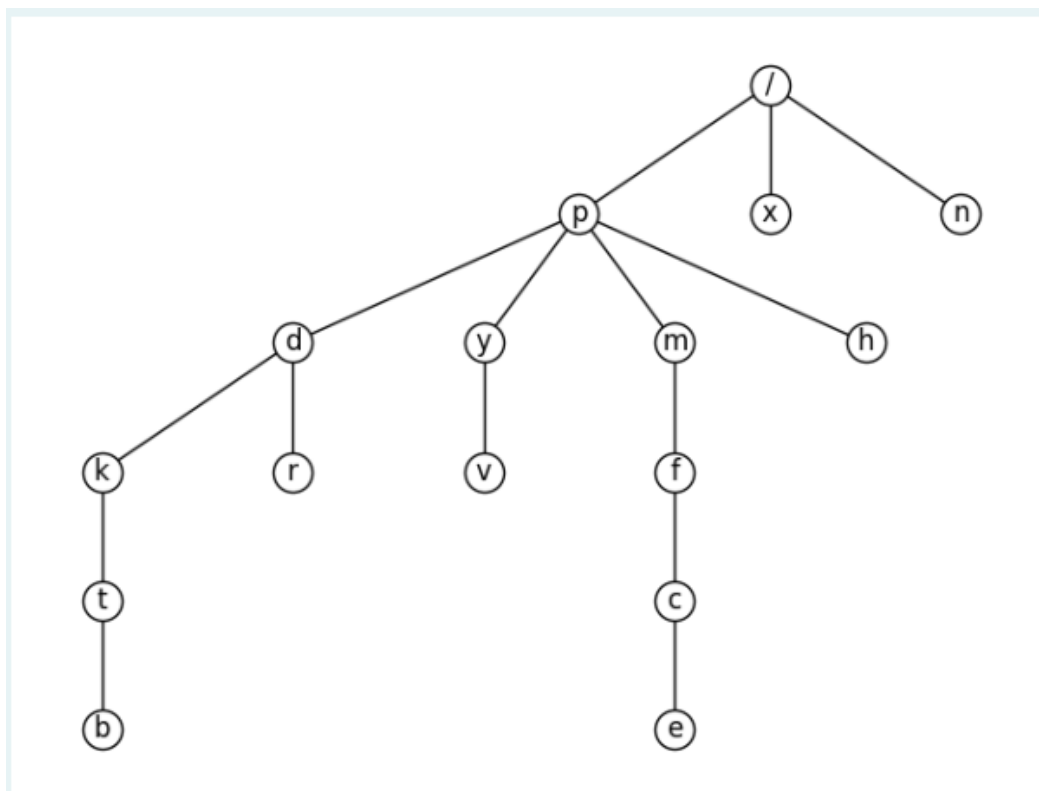
Se dă un planificator de tipul **Shortest Job First, varianta non-preemptivă**. Se lansează procesele de mai jos, pentru fiecare dându-se timpul de intrare în sistem (Arrival Time -- AT) și timpul total de rulare (Running Time -- RT).

P	AT	RT
1	3	2
2	5	4
3	1	6
4	0	3

Reprezentați diagrama Gantt corespunzătoare execuției, completând mai jos id-ul procesului activ care rulează pe procesor la fiecare moment de timp.

Deasemenea completați aici timpul mediu de așteptare (average waiting time) calculat: 3.2 ✖ [2] .

0	1 ✖ [4]
1	1 ✖ [4]
2	4 ✔
3	4 ✖ [1]
4	4 ✖ [1]
5	2 ✔
6	2 ✔
7	2 ✔
8	2 ✔
9	3 ✔
10	3 ✔
11	3 ✔
12	3 ✔
13	3 ✔
14	3 ✔



Se dă sistemul de fișiere din figura de mai sus. Puteți considera că toate nodurile sunt directoare.

La completarea căilor cerute mai jos, alegeți tot timpul forma cea mai simplă.

Completați calea **absolută** spre **k**: ✓

Presupunând că ne aflăm în directorul **/p/m**, completați calea **relativă** spre **c**: ✗ [f/c/?]

Presupunând că ne aflăm în directorul **/p/m/f/c**, completați calea **relativă** spre **m**: ✓

Presupunând că ne aflăm în directorul **/p/m/f**, completați calea **relativă** spre **t**: ✓